



Building a Quantum Machine Learning library in myQLM

Internship at Eviden (Atos Group)

Author: César Hernando de la Fuente

Company supervisor: Andrés Bravo Montes

University supervisor: Matthias Möller

28-10-2025



Agenda

1

Introduction

2

Problem
statement

3

QML algorithms
and results

4

Conclusion and
reflection on
learning



1

Introduction

1.1 Eviden Quantum

- Subsidiary of Atos focused on Cybersecurity, HPC, AI and **Quantum Computing**.
- Main offerings:
 - **myQLM** : quantum SDK to emulate quantum circuits + tutorials
 - **Qaptiva quantum emulation appliance**: larger emulation capabilities + noise + quantum optimizers
 - **Qaptiva HPC**: multi-node operation, compatible with Slurm
- **Consulting and support**
- **Quantum Computing Workshops**

1.1 Eviden Quantum

Quantum Machine Learning?



Running digital quantum simulations for spin and fermionic systems

Running digital quantum simulations for spin and fermionic systems



[overview](#)

Spin fermion transforms



Variational Quantum Eigensolver for Fermions: tutorial



Quantum Phase Estimation on the Hubbard molecule



Quantum chemistry: Solving H2 using UCC



Quantum chemistry: Solving LiH using UCC



Advanced VQE: Quantum Subspace Expansion



Natural gradient-based optimizer



Optimizing circuits with the sequential optimization plugin



Mitigating multiqubit gate noise



Combinatorial Optimization Problems

Combinatorial Optimization Problems



[overview](#)

Introduction



Max Cut



[beginner](#)

Graph Partitioning



[advanced](#)

K-Clique



[advanced](#)

Vertex Cover



[beginner](#)

Number Partitioning



[beginner](#)

Basic example of use of simulated annealing



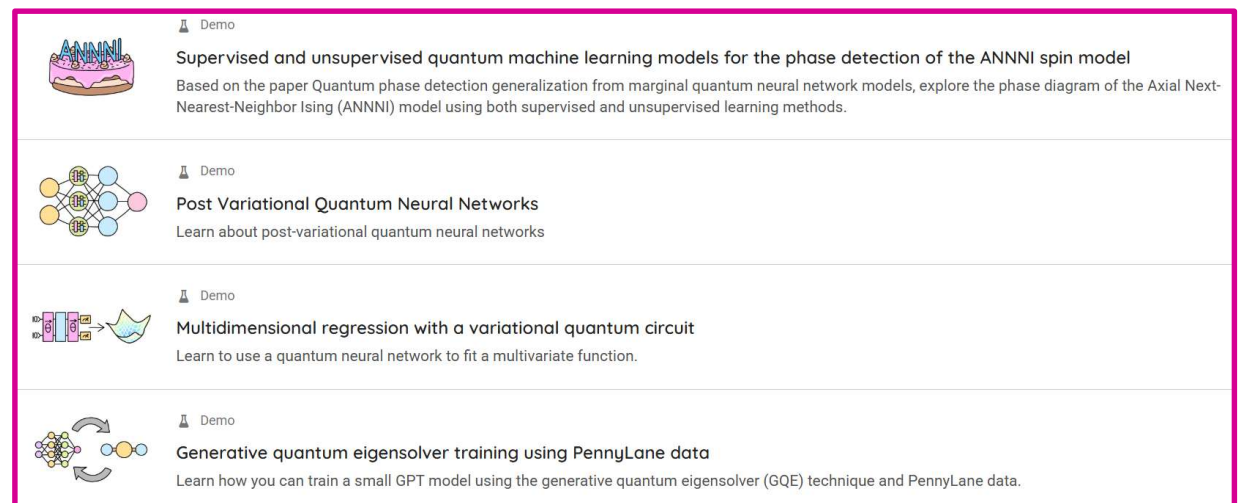
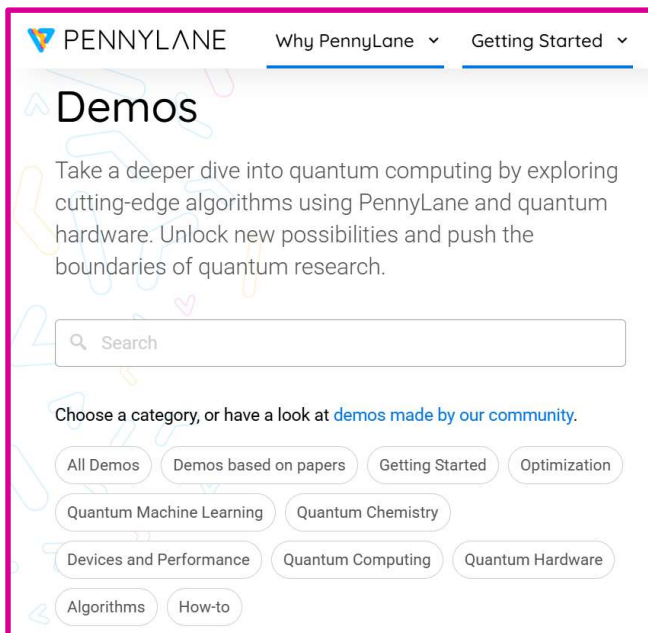
[beginner](#)

1.2 Quantum Machine Learning

- Quantum Machine Learning (QML) refers to machine learning algorithms that use quantum computing for certain subroutines with the goal of:
 - Achieving a reduction in computational complexity ➡ acceleration
 - Reducing the training data required to achieve a certain accuracy
 - Increasing the expressivity
- Reality: Quantum computers are noisy: limited coherence time and gate fidelities
- Theoretical advantage is not translated into practical usefulness
- However, research in QML is still needed for when quantum hardware catches up, e.g. deep learning.

1.2 Quantum Machine Learning

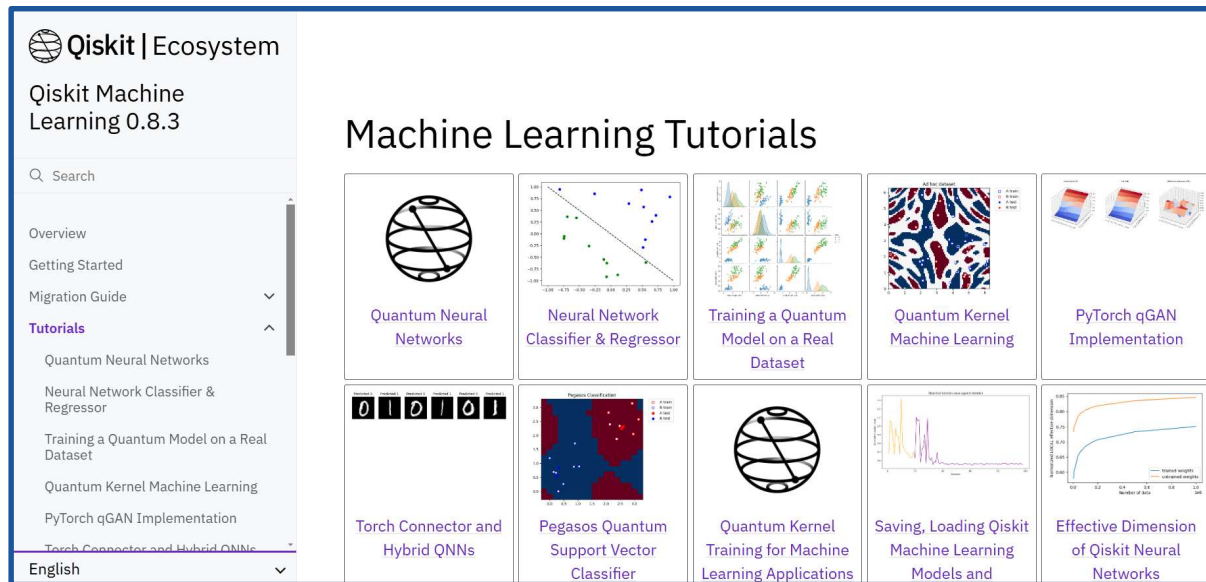
- Quantum computing companies such as **Xanadu**, **IBM** and **Google** have already built Quantum Machine Learning (QML) libraries in their SDKs (**PennyLane**, **Qiskit** and **Cirq**) as well as Jupyter Notebook demos



<https://pennyLane.ai/qml/demonstrations>

1.2 Quantum Machine Learning

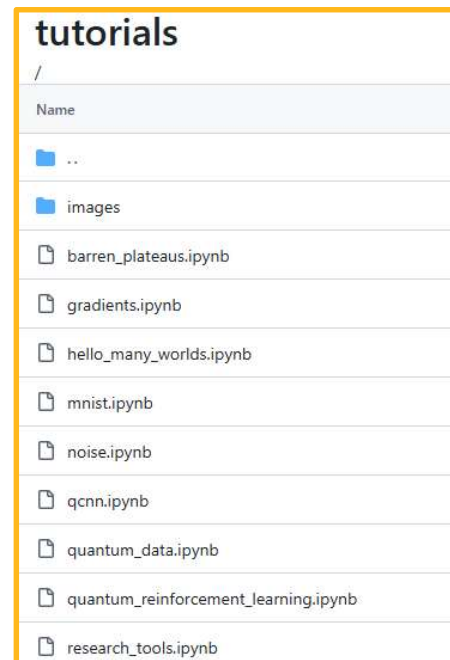
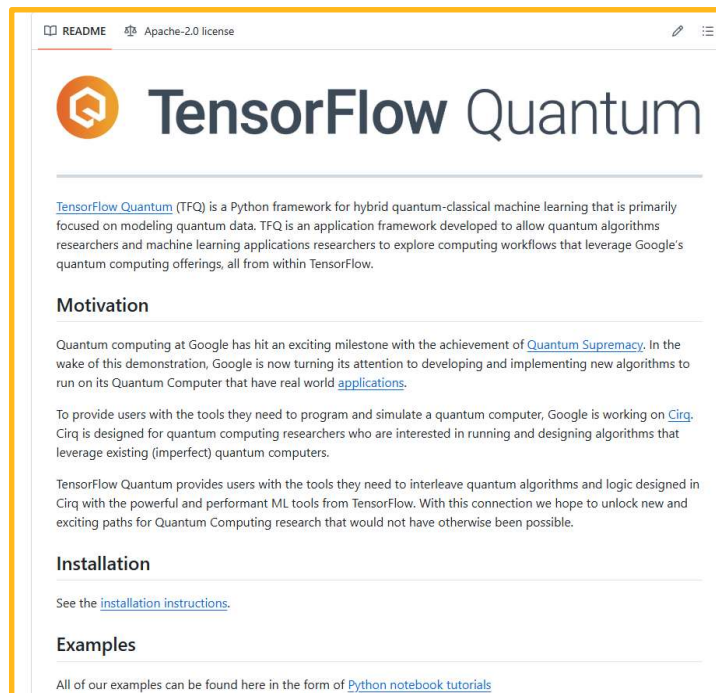
- Quantum computing companies such as **Xanadu**, **IBM** and **Google** have already built Quantum Machine Learning (QML) libraries in their SDKs (**PennyLane**, **Qiskit** and **Cirq**) as well as Jupyter Notebook demos



<https://qiskit-community.github.io/qiskit-machine-learning/tutorials/index.html>

1.2 Quantum Machine Learning

- Quantum computing companies such as Xanadu, IBM and Google have already built Quantum Machine Learning (QML) libraries in their SDKs (Pennylane, Qiskit and Cirq) as well as Jupyter Notebook demos



<https://github.com/tensorflow/quantum/tree/master/docs/tutorials>



2

Problem statement

2. Problem statement

- Develop a Quantum Machine Learning library in myQLM, together with Jupyter Notebook tutorials applying the algorithms to specific datasets
- This brings value to Eviden in several ways:
 1. Allow users to explore QML methods from a high-level perspective
 2. Make myQLM (Eviden) more competitive with respect to other QC companies
 3. Enable delivery of more advanced educational QC workshops
 4. Motivate the community to create and test their own QML routines, and develop demos that could be added to the official repository
- Additionally, I learn about this field and improve my research skills, useful for Master's thesis



3

QML algorithms

3. Quantum Machine Learning

There exists numerous QML algorithms. To include a few:

➤ Quantum Support Vector Machine (QSVM)

➤ Quantum Neural Networks / Parametrized Quantum Circuits (QNNs/PQCs)

➤ Quantum Convolutional Neural Networks (QCNNs)

➤ Quantum Circuit Born Machine (QCBM)

➤ Quantum Generative Adversarial Networks (QGANs)

➤ Quantum reinforcement learning for QEC Master's thesis

Supervised Learning

Unsupervised Learning

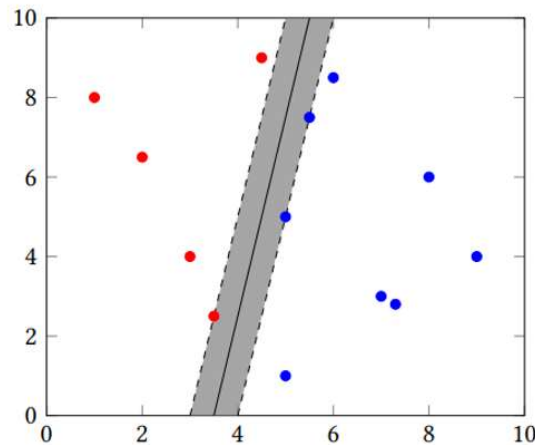
Reinforcement Learning

3.1

Quantum Support Vector Machine

3.1. Support Vector Machine: Binary classification

- Perhaps one of the simplest and easiest QML algorithm to implement
- The idea in a nutshell:



Hyperplanes equations $\vec{w} \cdot \vec{x} + b = \pm 1$.

Distance between hyperplanes $2/\|\vec{w}\|$

Hard-margin

$$\begin{aligned} &\text{Minimize} \quad \frac{1}{2} \|\vec{w}\|^2 \\ &\text{subject to} \quad y_j(\vec{w} \cdot \vec{x}_j + b) \geq 1. \end{aligned}$$



Soft-margin

$$\begin{aligned} &\text{Minimize} \quad \frac{1}{2} \|\vec{w}\|^2 + C \sum_j \xi_j \\ &\text{subject to} \quad y_j(\vec{w} \cdot \vec{x}_j + b) \geq 1 - \xi_j, \\ &\quad \quad \quad \xi_j \geq 0. \end{aligned}$$



Lagrangian dual problem

$$\begin{aligned} &\text{Maximize} \quad \sum_j \alpha_j - \frac{1}{2} \sum_{j,k} y_j y_k \alpha_j \alpha_k (\vec{x}_j \cdot \vec{x}_k), \\ &\text{subject to} \quad 0 \leq \alpha_j \leq C, \\ &\quad \quad \quad \sum_j \alpha_j y_j = 0. \end{aligned}$$



- Convex quadratic programming problem
- Solvable with Sequential Minimal Optimization (SMO)

3.1. Support Vector Machine

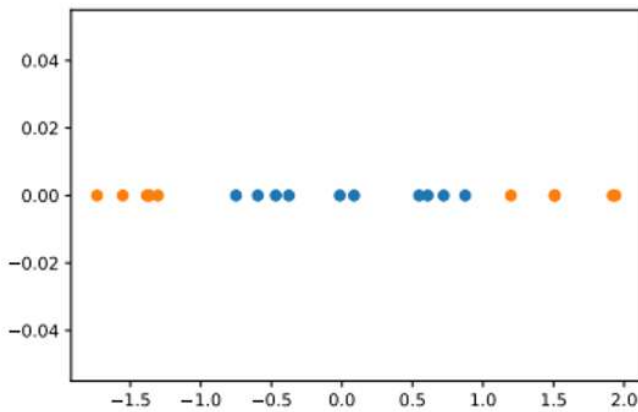
- With the current formulation, the algorithm could only be applied to problems where data is linearly separable
- Solution: Kernel trick → Map points $\mathbf{x}$ to points in a higher dimensional space, hoping that in that space (feature space) data can be separated linearly

$$\begin{aligned} &\text{Maximize} \quad \sum_j \alpha_j - \frac{1}{2} \sum_{j,k} y_j y_k \alpha_j \alpha_k (\vec{x}_j \cdot \vec{x}_k), \\ &\text{subject to} \quad 0 \leq \alpha_j \leq C, \\ &\quad \quad \quad \sum_j \alpha_j y_j = 0. \end{aligned}$$

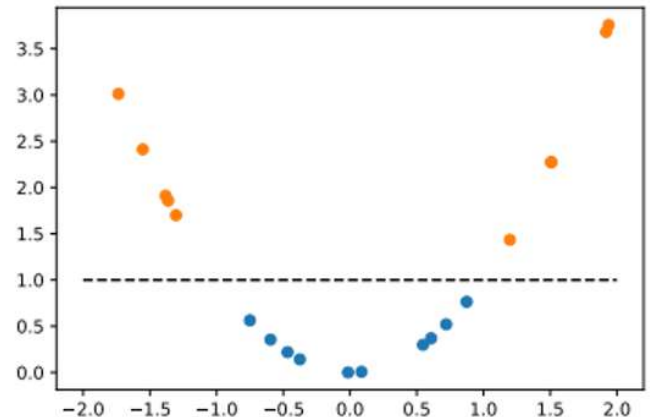


$$\varphi: \mathbf{x} \rightarrow \varphi(\mathbf{x})$$

$$k(\mathbf{x}, \mathbf{y}) = \varphi(\vec{x}) \cdot \varphi(\vec{y})$$



(a) Original data in the real line

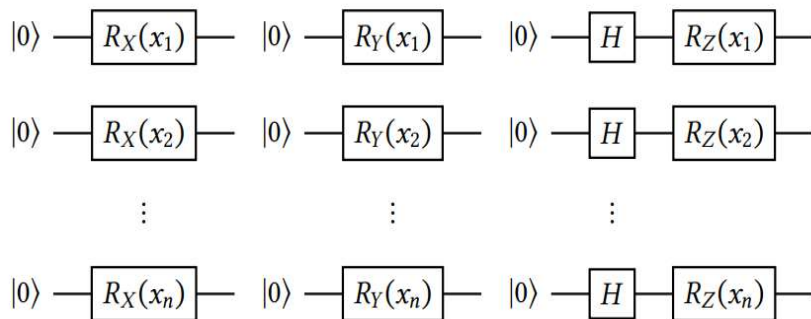


(b) Data in the feature space

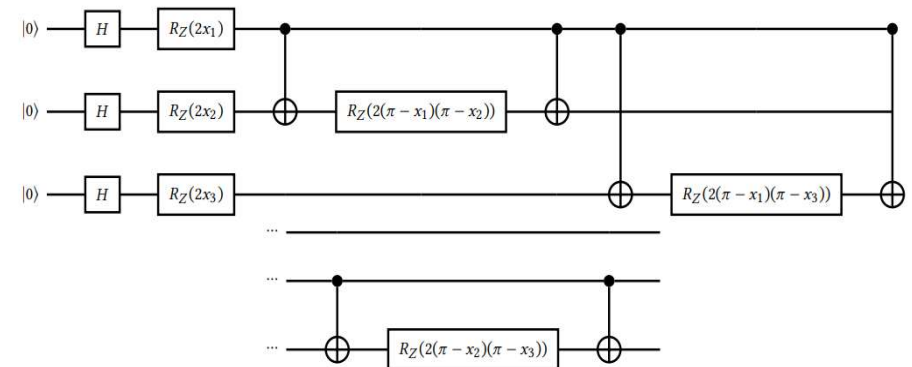
3.1. Quantum Support Vector Machine

- Going Quantum: Map points x to quantum states: $k(\vec{a}, \vec{b}) = |\langle \varphi(a) | \varphi(b) \rangle|^2 = \left| \langle 0 | \Phi^\dagger(a) \Phi(b) | 0 \rangle \right|^2$
- Hilbert space allows high expressivity
- There are many different types of feature maps, I implemented two:

Angle encoding



ZZ Feature map



3.1. Datasets and preprocessing

load_wine

```
sklearn.datasets.load_wine(*, return_X_y=False, as_frame=False) \[source\]
```

Load and return the wine dataset (classification).

Added in version 0.18.

The wine dataset is a classic and very easy multi-class classification dataset.

Classes	3
Samples per class	[59,71,48]
Samples total	178
Dimensionality	13
Features	real, positive

The copy of UCI ML Wine Data Set dataset is downloaded and modified to fit standard format from:
<https://archive.ics.uci.edu/ml/machine-learning-databases/wine/wine.data>

load_breast_cancer

```
sklearn.datasets.load_breast_cancer(*, return_X_y=False, as_frame=False) \[source\]
```

Load and return the breast cancer Wisconsin dataset (classification).

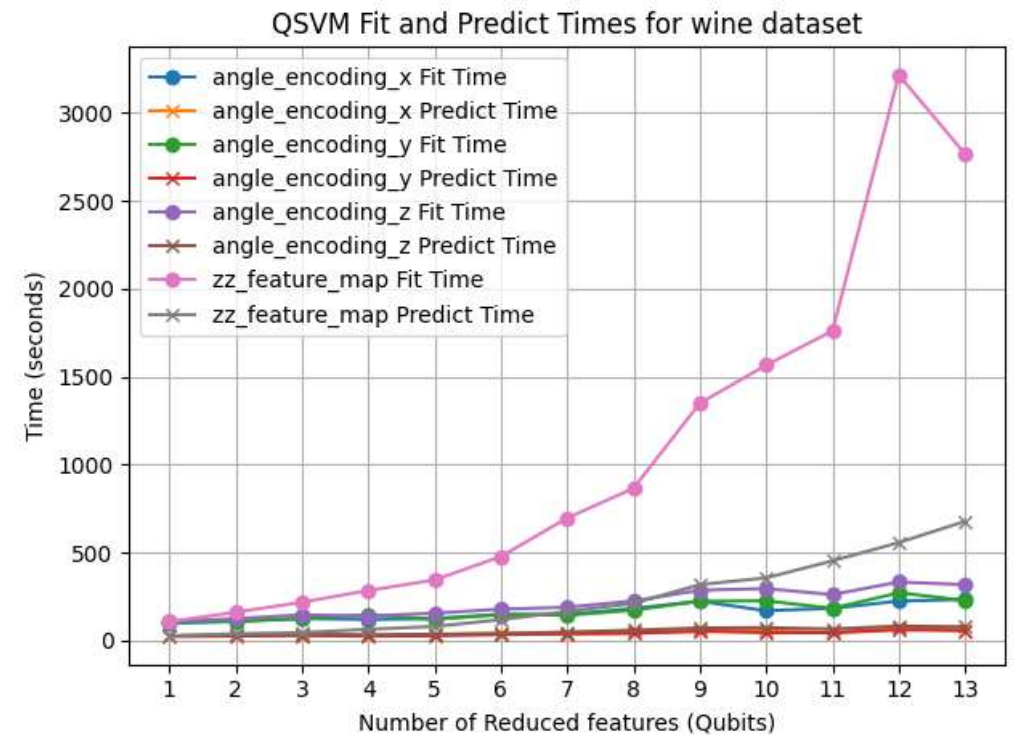
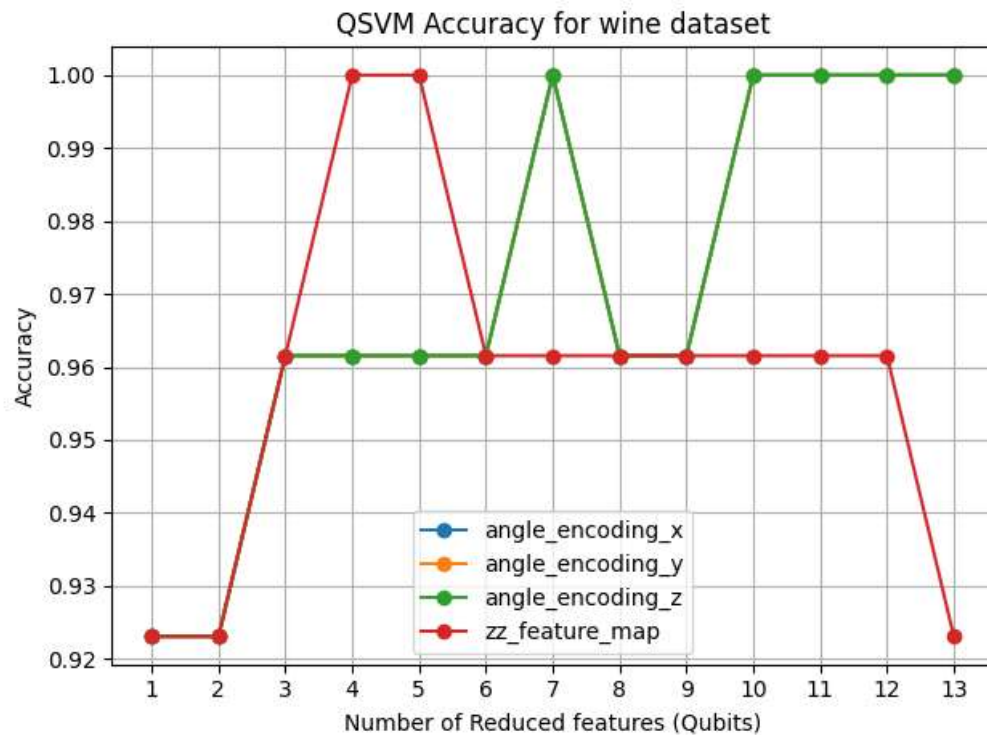
The breast cancer dataset is a classic and very easy binary classification dataset.

Classes	2
Samples per class	212(M),357(B)
Samples total	569
Dimensionality	30
Features	real, positive

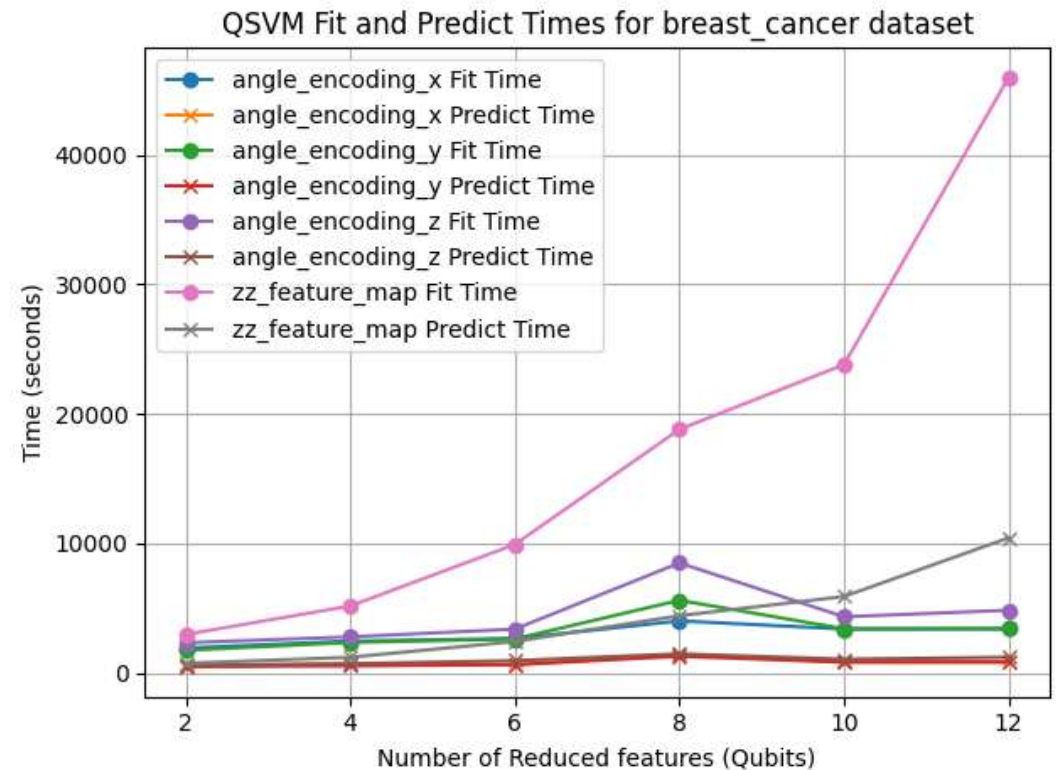
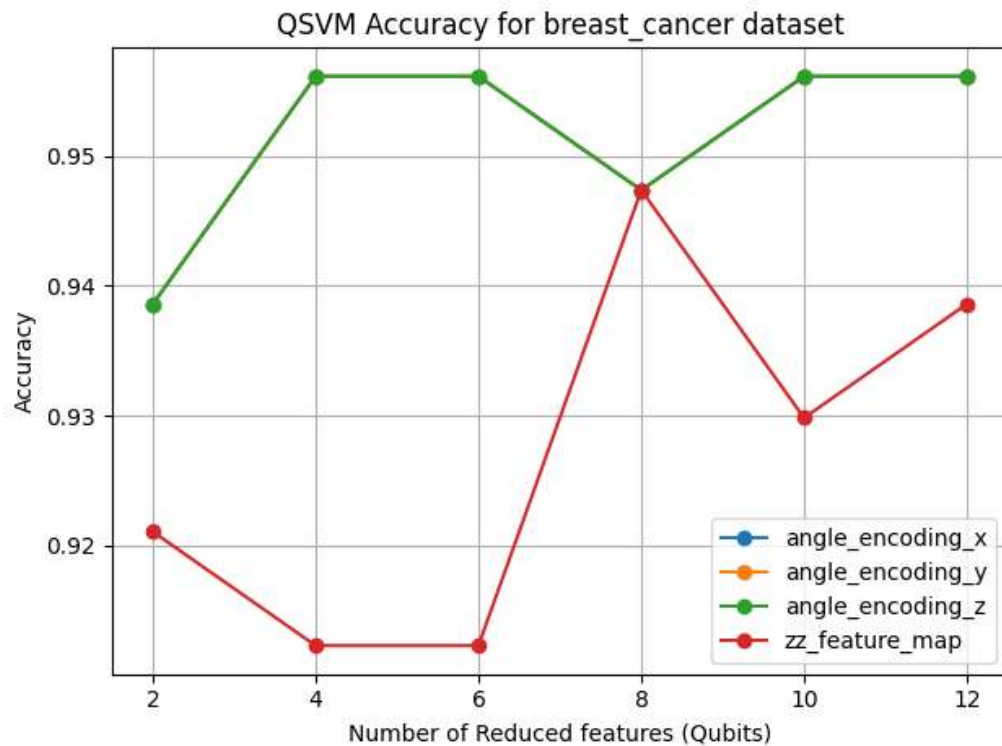
The copy of UCI ML Breast Cancer Wisconsin (Diagnostic) dataset is downloaded from:
<https://archive.ics.uci.edu/dataset/17/breast+cancer+wisconsin+diagnostic>

- Split the data in training and test, normalize the features (using only training data to prevent data leakage)
- To reduce the computation time, we perform dimensionality reduction using Principal Component Analysis (PCA)

3.1. Results for wine dataset



3.1. Results for breast cancer dataset



3.1. Conclusions

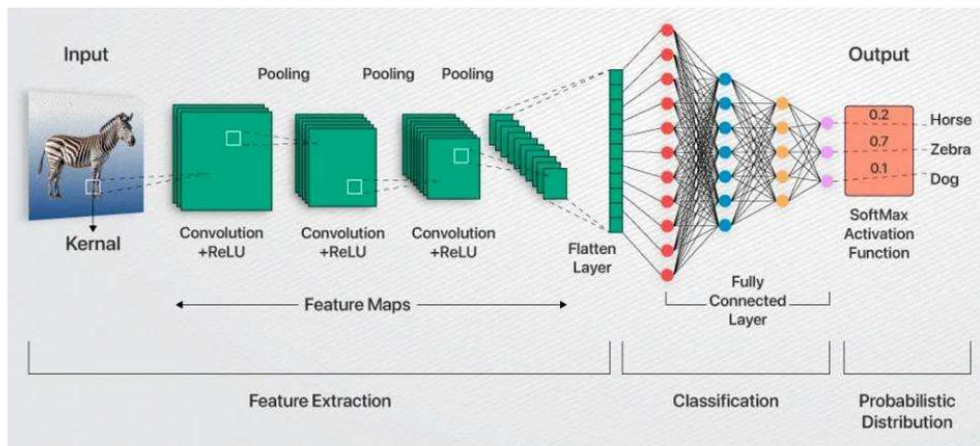
- Angle encoding achieves a higher accuracy and has a better computational time scaling compared to ZZ feature map
- The rotation axis is irrelevant in terms of accuracy, but the rotation around Z has a higher computational cost due to the Hadamard gates
- Both datasets show a non-monotous dependence with the number of PCA-reduced features
- QSVM learns to predict the corresponding label with high accuracy for both datasets

3.2

Quantum- Classical CNNs

3.2. Quantum Convolutional Neural Networks

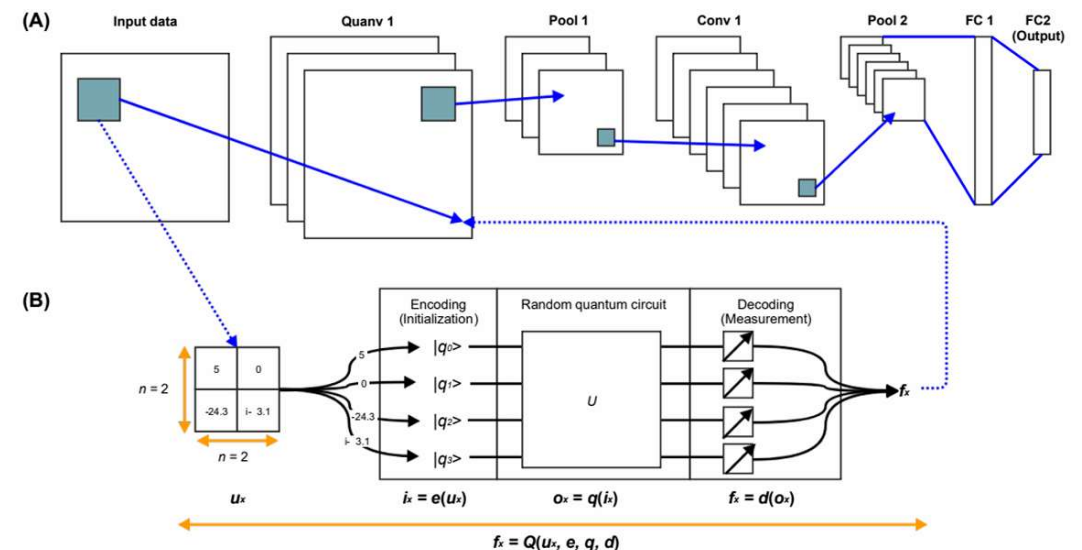
Classical CNN



- Achieved superhuman performance in complex visual tasks such as image search services, automatic video classification systems, and even self-driving cars

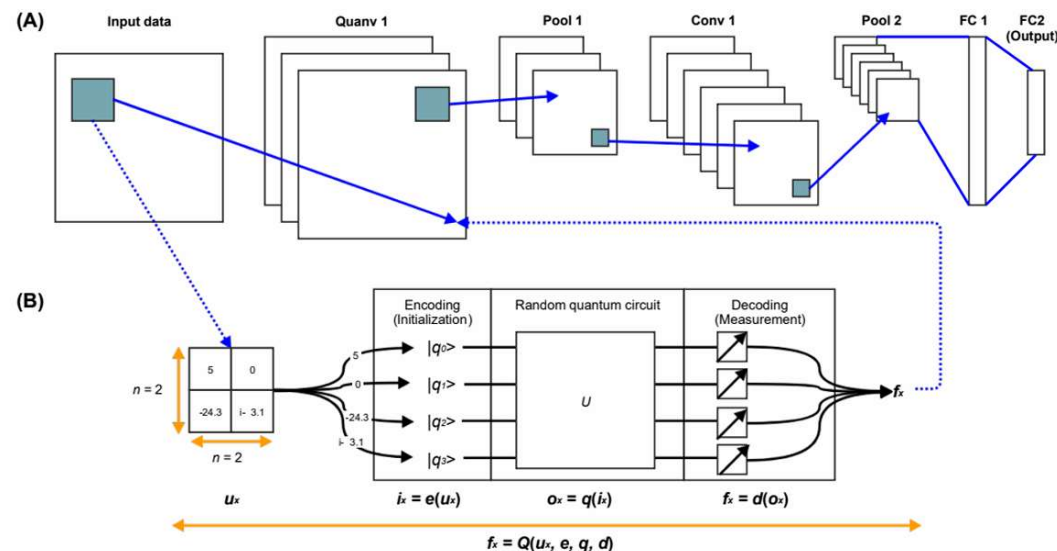
Quanvolutional Neural Network:

Maxwell Henderson et al. "Quanvolutional neural networks: powering image recognition with quantum circuits"



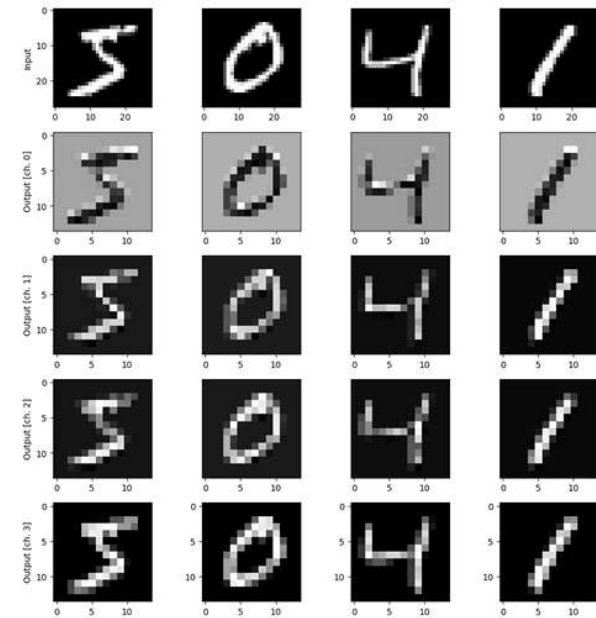
3.2. Quantum Convolutional Neural Networks

- Instead of linear transformation + activation function, use feature map + random quantum circuit + measurement
- Each of the qubit output (expectation value of a certain observable) goes to a pixel of a different channel in the next layer
- Quantum layer acts as a preprocessing since it does not have trainable parameters. How quantum is this?

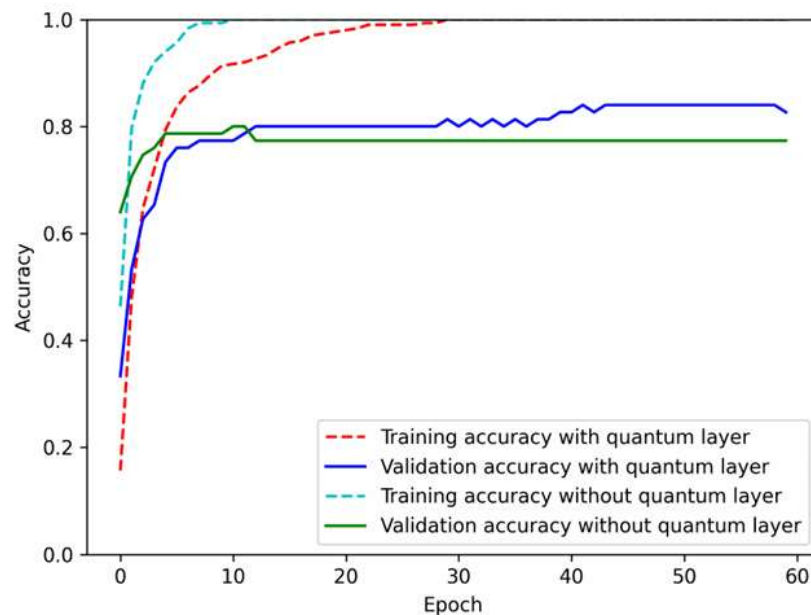


3.2. Quantum Convolutional Neural Networks

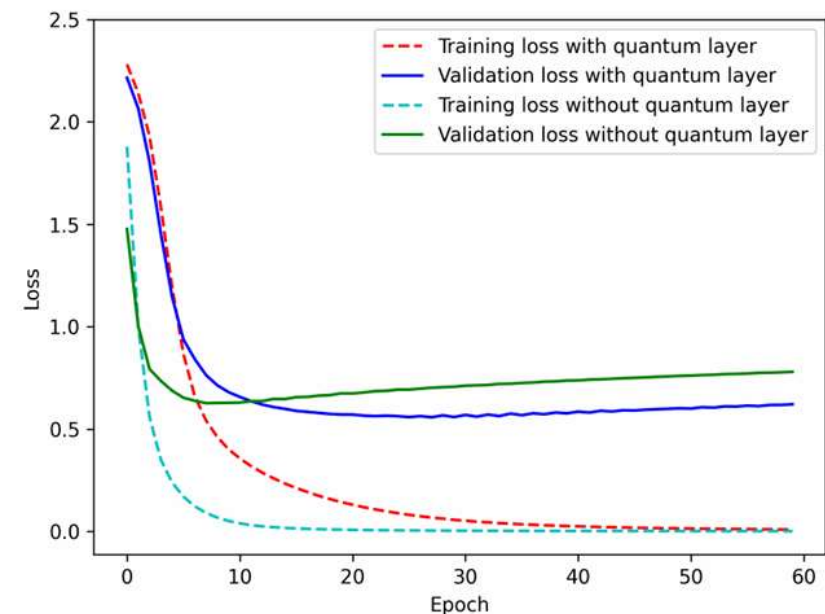
- We apply this architecture to the MNIST dataset: 28x28 gray scale images of hand-written digits, along with a label of the digit they represent.
- 300 images for training, 75 for testing
- Angle encoding feature map
- Hardware efficient ansatz of depth 3 as random circuit
- Only one classical CNN layer with 15 filters of size 2x2
- Max pooling (2x2)
- Fully connected (64 neurons)
- Adam optimizer
- 60 epochs, batches of 16, learning rate of 0.001
- Categorical cross-entropy loss function



3.2. Quantum Convolutional Neural Networks



(a) Training and validation accuracy.



(b) Loss: Sparse categorical cross entropy.

Figure 10: Comparison of performance when the quantum convolutional layer is applied against when it is not applied.

3.2. Conclusions

- Both models memorize the data perfectly, as the training accuracy reaches 100%, but the quantum model takes more epochs to do it.
- This is reasonable, as the quantum model is slightly more complex, as the input images have four channels instead of one.
- From the moment the training accuracy of the classical model reaches to 100%, the validation accuracy reaches to a plateau at around 77%, and it slightly overfits.
- In contrast, the quantum model validation accuracy continuous increasing up to 84%. Thus, adding the quanvolutional layer increases the validation accuracy 7%.
- However, this improvement comes with a significant overhead in computation time.
- Adding an extra classical convolutional layer with higher number of filters can improve the accuracy to 96%.

3.3

Quantum Circuit Born Machine

3.3. Generative unsupervised learning

Discriminative supervised learning

- Given a dataset $D = \{(x_i, y_i) | x_i \in S; y_i \in \text{labels}\}$, the task is to learn the label function $f: S \rightarrow \text{labels}$ that predicts the label for “seen” (training data) and “unseen” data as “good” as possible.
- Probabilistic interpretation: Given some data and labels sampled from an underlying probability distribution: $D \sim P^{\times |D|}$, learn a good guess for $P(y|x)$.

Generative unsupervised learning

- Given some data $x \in S$, learn the underlying probability distribution $P(x)$ and generate new data by sampling from it.
- In simple terms, these algorithms learn the common features of the data and generate new samples that resembles the original data.

There are also discriminative unsupervised algorithms (clustering, e.g. K-means, DBSCAN) and other types of unsupervised algorithms (dimensionality reduction/data compression, e.g. PCA, Autoencoder NN)

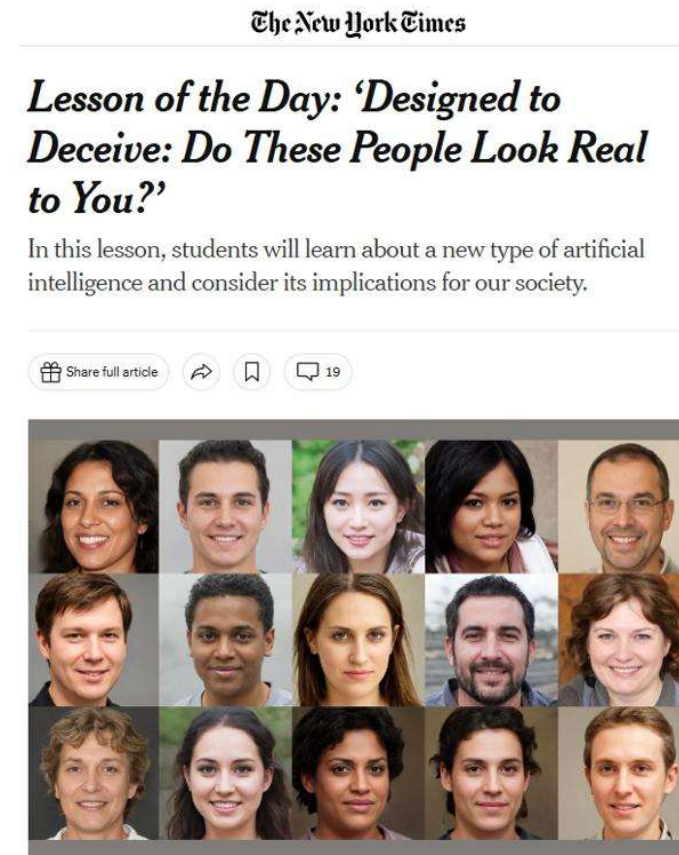
3.3. Generative unsupervised learning

Examples of applications

- Given a dataset of cats. Generate new cats
- Generate fake pictures of people
- Drug discovery by generating new molecular structures

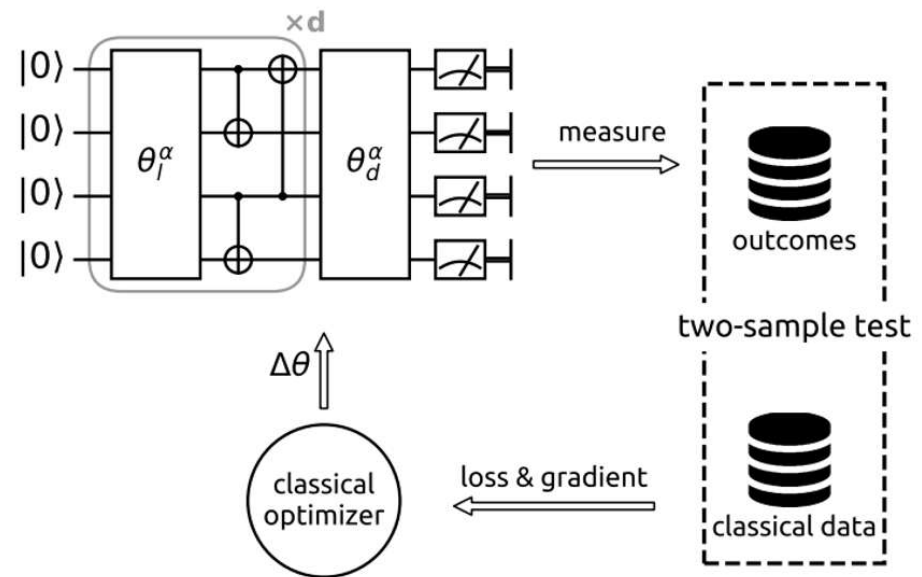
Most important classical ML algorithms

- Variational Autoencoders (VAE)
- Generative Adversarial Networks (GANs)
- Diffusion models
- Restricted Boltzmann Machine



3.3. Quantum Circuit Born Machine

- Generative model which represents/encodes the probability distribution of classical data as quantum pure states.
- It is based on the Born's postulate: $P(x) = |\langle x | \psi_\theta \rangle|^2$.
- Sampling is efficient, as it corresponds to circuit runs (one shot for one sample)
- Uses the Squared maximum mean discrepancy (MMD) loss function with a mixture Gaussian kernel.
- It is a gradient-based algorithm, which uses classical optimizers like Adam.



Jin-Guo Liu and Lei Wang (2018). Differentiable learning of quantum circuit Born machines . *Physical Review A*, <https://doi.org/10.1103/PhysRevA.98.062324>

3.3. Quantum Circuit Born Machine

- In the same spirit as in Supervised QNN, where parametrized quantum circuits (with rotation gates) are used, the gradient of the loss function with respect to the los function can be calculated using the Parameter Shift Rule:

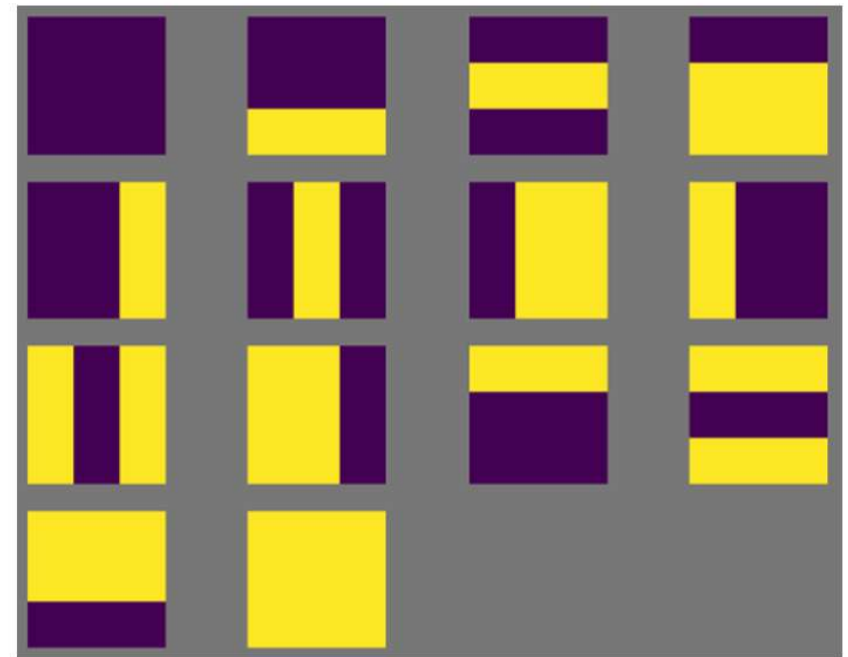
$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial \theta_l^\alpha} = & \mathbb{E}_{x \sim p_{\theta^+}, y \sim p_\theta} [K(x, y)] - \mathbb{E}_{x \sim p_{\theta^-}, y \sim p_\theta} [K(x, y)] \\ & - \mathbb{E}_{x \sim p_{\theta^+}, y \sim \pi} [K(x, y)] + \mathbb{E}_{x \sim p_{\theta^-}, y \sim \pi} [K(x, y)]. \end{aligned} \quad (2)$$

Here, $p_{\theta^+}(x)$ and $p_{\theta^-}(x)$ are output probabilities of QCBM under circuit parameters $\theta^\pm = \theta \pm \frac{\pi}{2} \mathbf{e}_l^\alpha$, where $\mathbf{e}_l^\alpha$ is the (l, α) -th unit vector in parameter space (i.e. $\theta_l^\alpha \leftarrow \theta_l^\alpha \pm \frac{\pi}{2}$, with other angles unchanged). In contrast to the finite difference methods like simultaneous perturbation stochastic approximation (SPSA) [58], Eq. (2) is an unbiased estimator of the exact gradient. Its detailed derivations is in Appendix A.

Jin-Guo Liu and Lei Wang (2018). Differentiable learning of quantum circuit Born machines . *Physical Review A*, <https://doi.org/10.1103/PhysRevA.98.062324>

3.3. Quantum Circuit Born Machine

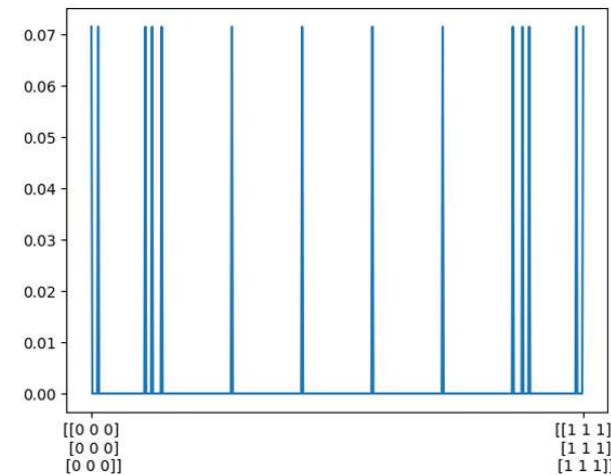
- The Bars and Stripes of size N dataset consists of the matrices $N \times N$ that satisfy one of the following conditions (or both):
 1. For every column, all elements are equal (Bars).
 2. For every row, all elements are equal (Stripes).
- There are two goals:
 1. Learn (memorize) the probability distribution and sample valid matrices.
 2. Given only a subset of valid matrices for training, sample unseen valid matrices
⇒ generalization and generative capability



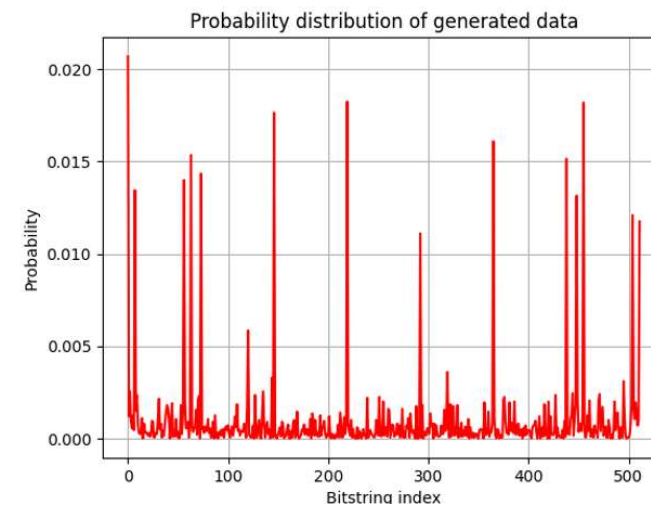
3.3. Quantum Circuit Born Machine

- 9 qubits
- 10 blocks
- 20000 shots
- $\text{sigma_list_kernel} = [0.5, 1, 2, 4]$
- Adam optimizer (learning rate = 0.1)
- We use all the valid matrices for training
- 10000 generated samples

Target distribution



Generated data



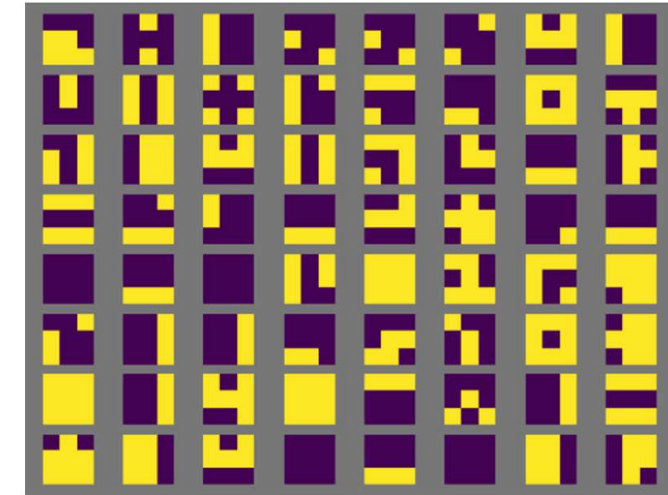
3.3. Quantum Circuit Born Machine

Precision: $p = \frac{\# \text{ valid samples}}{\# \text{ generated samples}} = 0.42$

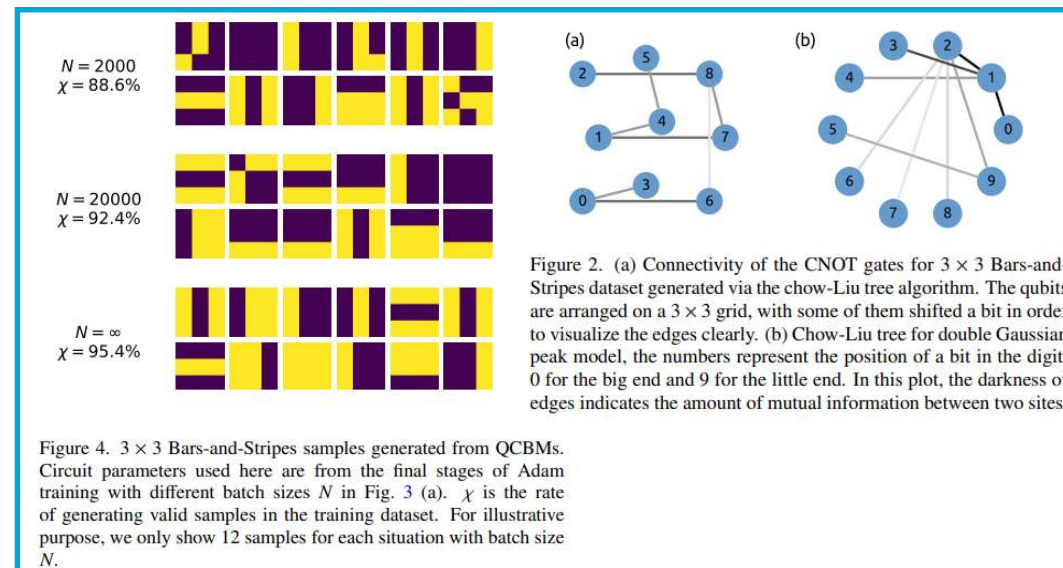
Average cost = 1.23

The cost measures how off a sample is from being a Bar or a Stripe, i.e. how many pixels we would have to change to get a Bar or Stripe

64 first generated samples



Paper:



3.3. Quantum Circuit Born Machine

Do Quantum Circuit Born Machines Generalize?

Kaitlin Gili,^{1,2} Mohamed Hibat-Allah,^{2,3,4} Marta Mauri,² Chris Ballance,¹ and Alejandro Perdomo-Ortiz^{2,*}

¹University of Oxford, Oxford, United Kingdom OX1 2JD

²Zapata Computing Canada Inc., 325 Front St W, Toronto, ON, Canada M5V 2Y1

³Vector Institute, MaRS Centre, Toronto, ON, Canada M5G 1M1

⁴Department of Physics and Astronomy, University of Waterloo, Waterloo, ON, Canada N2L 3G1

(Dated: May 16, 2023)

In the paper, the following generalization metrics are defined:

- $Fidelity = \frac{\# \text{ valid and unseen samples}}{\# \text{ unseen samples}}$

describes how well the model can generalize to valid samples rather than produce noise; and the coverage

- $Rate = \frac{\# \text{ valid and unseen samples}}{\# \text{ generated samples}}$

describes the frequency at which the model generates unseen valid samples

- $Coverage = \frac{\# \text{ unique valid and unseen samples}}{\# \text{ solutions} + \# \text{ training samples}}$

describes the portion of the unseen valid space the model is able to learn



4

Conclusions and reflection on learning

General conclusions

- The results pointed out that although the three QML algorithms were correctly implemented, this area is still far from being useful for real datasets, due to its high computational time when running in simulators.
- It is still important to design new QML algorithms for the future, when the quantum hardware is more mature
- However, the main goal of this internship was not assessing QML algorithms performance, but creating a library and Jupyter Notebooks tutorials for users and developers, in order to help them understand this new popular area of research.

Extra contributions

- I worked together with another intern (Daniel Maroto) on a study of the main quantum simulators integrable in HPC clusters. We compared their different characteristics, such as CPU multi-core and multi-node compatibility, GPU acceleration, job scheduler integration, support service, and many others.
- This study was useful to the department in their write-up of a small section of a research paper.
- Additionally, I helped another intern of the team with his project on hyperparameter tuning of the Simulated Quantum Annealing (SQA) algorithm, implementing a Bayesian Optimization routine.

Reflection on learning

- Learnt about some Quantum Machine Learning algorithms
- Indirectly, I also learnt Classical Machine Learning algorithms and data analysis methods
- Improved my Python Object-Oriented Programming skills
- Research and paper reading skills
- Learnt about the department's business solutions, realizing all the work is necessary in a project apart from the implementation itself.

Thank you for your attention!

Questions?